

Quantitative Large Language Models

A Proposal for Physics-Informed Mathematical Reasoning at Scale

December 2025

Executive Summary

Current large language models excel at verbal reasoning but fundamentally lack quantitative intelligence. They manipulate symbols representing mathematics without performing genuine mathematical operations. This proposal outlines a transformative approach: building quantitative LLMs that integrate physics-informed mathematical frameworks as computational primitives, enabled by a novel Event Processing Unit (EPU) hardware architecture designed specifically for constraint-driven computation.

The foundation exists in a validated mathematical framework based on complete elliptic integrals computed via the Arithmetic-Geometric Mean (AGM). This framework demonstrates:

- Computational efficiency: 5 iterations to machine precision, delivering 10^5 – $10^6\times$ speedup versus traditional PDE solvers
- Conservation by construction: Transfer function formulation automatically enforces physical laws
- Novel hardware substrate: Event Processing Unit (EPU) architecture provides native support for constraint-driven computation with 10-100× power efficiency and 10× lower latency versus GPU/NPU for synchronization operations
- Domain agnosticism: Validated on energy systems with clear pathways to biological, manufacturing, and financial applications
- Universal invariants: Extracts continuous (ξ) and discrete (S) correlates that align with experimental observables
- Scale-free field: Maps physical parameters to a normalized range (0 to 1) bridging abstraction and physical reality

This proposal advocates for integrating such physics-informed mathematical kernels into LLM architectures, creating models that perform authentic quantitative reasoning rather than pattern-matching numerical text.

I. The Fundamental Limitation of Current LLMs

The Verbal Ceiling

Large language models process mathematics as linguistic artifacts—symbols arranged according to learned patterns. Consider the task of computing eigenvalues for a 100×100 matrix:

Current LLMs: Generate text describing eigenvalue algorithms, potentially write code that calls numerical libraries, or approximate solutions based on pattern recognition.

Quantitative LLM: Execute native mathematical operations—actually compute eigenvalues using built-in numerical kernels that respect matrix theory and convergence properties.

This distinction becomes critical in scientific domains where conservation laws, physical constraints, and numerical precision are non-negotiable.

The Cost of Abstraction

Modern scientific computing faces three persistent challenges:

1. **Computational Intractability:** High-dimensional PDEs governing fluid dynamics, biochemical networks, or financial derivatives require prohibitive computation (10^7 – 10^9 operations per evaluation)
2. **Conservation Violations:** Pure ML approaches violate fundamental constraints (mass balance, energy conservation, stoichiometry) and fail to extrapolate beyond training data
3. **Interpretability Loss:** Black-box neural networks provide no mechanistic understanding, making them unsuitable for safety-critical applications or regulatory environments

Current LLMs cannot address these challenges because they lack native quantitative reasoning capabilities. They can discuss conservation laws but cannot enforce them computationally.

II. Vision: Quantitative LLMs

Defining Quantitative Intelligence

A quantitative LLM integrates three distinct reasoning modes:

Mode	Capability	Example
Verbal	Pattern recognition, linguistic reasoning, knowledge synthesis	Explain Maxwell's equations conceptually
Symbolic	Algebraic manipulation, calculus operations, theorem proving	Derive wave equation from Maxwell's equations
Numerical	Compute solutions respecting conservation laws and physical constraints	Solve electromagnetic field distribution in specific geometry

Current LLMs excel at verbal mode, show emerging capability in symbolic mode, but completely lack numerical mode. Quantitative LLMs would integrate all three.

The Architecture Gap

Transformer architectures process discrete tokens—fundamentally unsuited for continuous numerical computation. Even when LLMs generate code that calls numerical libraries, they remain external to the mathematical operations. The model cannot learn from the computation itself.

A quantitative LLM architecture would embed mathematical kernels as primitive operations, analogous to how attention mechanisms are primitives in transformers. The model would perform actual computations, not simulate them through text generation.

III. Technical Foundation: AGM-Based Framework

Mathematical Architecture

The proposed quantitative kernels build on a validated framework employing complete elliptic integrals computed via the Arithmetic-Geometric Mean (AGM). This framework provides a concrete blueprint for physics-informed numerical reasoning:

Core Algorithm

The algorithm maps physical parameters to transfer functions:

$$H(s) = k \prod (s - z_i) / \prod (s - p_j)$$

where zeros (z_i), poles (p_j), and gain (k) emerge from elliptic integral evaluation with arguments derived from system characteristics (temperature, computational time, hierarchical structure).

The AGM algorithm converges quadratically:

$$a_i = (a_{i-1} + b_{i-1})/2, \quad b_i = \sqrt{a_{i-1} \times b_{i-1}}$$

This achieves machine precision in approximately 5 iterations, regardless of system dimensionality—a computational efficiency unmatched by iterative PDE solvers.

Conservation by Construction

The transfer function formulation inherently enforces three critical properties:

1. **Passivity:** Gain scaling prevents violation of mass/energy conservation
2. **Stability:** All poles in left half-plane ensure system returns to equilibrium
3. **Causality:** Proper pole-zero structure respects temporal ordering

These constraints are not imposed as soft penalties during training—they are guaranteed by mathematical structure. This represents a fundamental advantage over purely data-driven approaches where conservation violations are inherent risks.

Universal Invariants

The framework extracts two independent mathematical invariants from each transfer function:

ξ (continuous): Log-deviation from baseline: $\xi = \ln(\Lambda S / \Lambda G)$, creating a scale-free field ranging from 0 to 1

S (discrete): Topological parity: difference between pole and zero counts, capturing structural system state

These invariants demonstrably align with experimental observables across validated test cases, confirming their physical interpretability and predictive power.

Validated Performance

The framework has been rigorously validated on biomass gasification systems (200+ experimental cases), achieving:

- Mass closure without explicit PDE solving—the algorithm reproduces conservation equation outcomes constructively
- Computational speedup of 10^5 – $10^6\times$ versus traditional CFD approaches

- Physical scale alignment: gain values ($k \sim 10^{-47}$ to 10^{-60}) numerically proximate to cosmological constant scales
- Mechanistic correlation: continuous (ξ) tracks product yield, discrete (S) correlates with conversion efficiency

This empirical validation demonstrates that special-function-parameterized approaches can capture essential physics across scales and domains, providing confidence for broader application.

IV. Computational Substrate: Event Processing Unit (EPU)

The Hardware Bottleneck

Quantitative LLMs require a fundamentally different computational substrate than current AI accelerators. While GPUs excel at dense matrix multiplication and NPUs optimize tensor operations, the constraint-driven nature of physics-informed computation demands native support for:

- **Binary state transitions:** Conservation constraints manifest as boolean flags (satisfied/violated)
- **Synchronization primitives:** Multiple concurrent inference threads must coordinate on shared constraint states
- **Event-driven computation:** State changes trigger cascading updates through dependency graphs
- **Multi-domain coordination:** Distinct physical domains (mass, energy, momentum) operate as concurrent state machines

Current hardware architectures handle these operations inefficiently, treating them as software constructs layered atop arithmetic-optimized silicon. This architectural mismatch creates unnecessary overhead precisely where quantitative LLMs need maximum efficiency.

EPU: A Binary State Machine Architecture

The Event Processing Unit (EPU) represents to constraint-driven computation what the GPU represents to parallel arithmetic: a fundamental architectural shift designed for a new class of workloads.

The EPU is not a faster CPU or a specialized GPU—it is a purpose-built substrate where binary state transitions, synchronization, and constraint enforcement are native hardware operations.

Architectural Principles

The EPU architecture comprises five interconnected layers:

Layer 1: Event Register File

Massive arrays of single-bit registers organized by domain (mass conservation events, energy balance events, causality constraints). Unlike general-purpose registers, event registers broadcast state changes to dependent computation units.

Layer 2: State Transition Logic (STL)

Hardware implementation of SET, CLEAR, TEST, and WAIT operations with sub-nanosecond latency. The instruction set natively expresses constraint logic rather than requiring compilation to arithmetic operations.

Layer 3: Gate Enforcement Hardware

Dedicated silicon for evaluating constraint predicates and blocking invalid state transitions at the hardware level. Conservation violations cannot propagate—they are architecturally impossible.

Layer 4: Event Dispatch Network (EDN)

High-speed interconnect broadcasting state changes to dependent cores. When a mass balance constraint is satisfied, all cores waiting on that event receive notification in a single cycle—no polling, no cache coherence overhead.

Layer 5: Provenance Hardware Buffer

Automatic tracking of which events triggered which state transitions, enabling interpretability and debugging at hardware speed. Critical for scientific applications requiring audit trails of constraint satisfaction.

Performance Characteristics

The EPU's architectural specialization delivers dramatic advantages for constraint-driven workloads:

Metric	GPU	NPU	EPU
Optimized for	FP×FP operations	Tensor operations	State transitions
Native operation	a+b, a×b	GEMM	SET, CLEAR, WAIT
Data width	32-64 bits	8-16 bits	1 bit
Power per op	1-10 pJ	0.1-1 pJ	0.01 pJ
Context switch	1000 cycles	500 cycles	50 cycles
Single op latency	~10 ns	~10 ns	~1 ns
Synchronization	Global memory	Shared memory	Event broadcast
Quantitative LLM fit	Moderate	Good	Excellent

The EPU delivers 10-100× lower power per operation while achieving 10× lower latency for constraint evaluation—precisely the operations dominating quantitative LLM inference.

EPU-Enabled Quantitative Inference

Consider the computational profile of quantitative LLM inference on a typical scientific problem:

- Problem encoding (transformer):** ~60% of operations—handled by NPU tensor cores
- AGM-elliptic integral evaluation:** ~10% of operations—handled by specialized floating-point units
- Constraint checking and synchronization:** ~25% of operations—currently inefficient on GPU/NPU
- Multi-domain state coordination:** ~5% of operations—massive overhead on conventional architectures

On a GPU/NPU hybrid system, constraint operations require expensive memory synchronization and software locks. On an EPU, these same operations execute as native hardware primitives:

GPU/NPU: 1000 cycles (memory barrier + lock acquire + state check)

EPU: 50 cycles (hardware WAIT on event register)

This 20× reduction in synchronization overhead directly translates to reduced latency for concurrent inference workloads—exactly the scenario when deploying quantitative LLMs in production environments handling multiple simultaneous scientific queries.

Hybrid EPU-NPU Architecture

The optimal quantitative LLM system employs a heterogeneous architecture:

- **NPU clusters:** Handle transformer attention mechanisms, embedding layers, and dense feature extraction
- **Specialized math units:** Execute AGM iterations and elliptic integral evaluation with high numerical precision
- **EPU array:** Coordinate constraint satisfaction, manage multi-domain state machines, enforce conservation laws
- **High-speed interconnect:** Event Dispatch Network links EPU cores with minimal latency event propagation

This heterogeneous design mirrors the hybrid nature of quantitative reasoning itself: linguistic (NPU), numerical (math units), and constraint-driven (EPU)—each architectural component optimized for its domain.

Training Versus Inference Considerations

The EPU architecture is initially optimized for inference—the deployment scenario where constraint checking dominates. However, training quantitative LLMs also benefits from EPU capabilities:

Gradient computation under constraints:

During backpropagation, gradients must respect conservation law boundaries. EPU hardware gates prevent invalid gradient updates, ensuring training never explores physically impossible regions of parameter space.

Multi-sample batch coordination:

Training on diverse physical scenarios requires coordinating constraint satisfaction across batch elements. EPU event broadcast enables efficient synchronization of constraint states across training samples.

Validation during training:

Real-time constraint verification during training iterations catches architectural issues immediately rather than discovering conservation violations only at evaluation time.

While NPU tensor operations remain the computational workhorse for training, EPU augmentation transforms constraint enforcement from a software validation step into a hardware-guaranteed property of the training process.

V. Quantitative LLM Architecture Design

Hybrid Architecture Principles

A quantitative LLM integrates transformer-based linguistic processing with mathematical kernel layers. The architecture embodies three design principles:

Principle	Implementation
Primitive Operations	Embed AGM, elliptic integral, transfer function evaluation as native layer operations alongside attention mechanisms
Differentiable Physics	Ensure all mathematical kernels support automatic differentiation for end-to-end gradient-based learning
Constraint Preservation	Build conservation laws into architecture rather than imposing as training objectives—guarantees physical validity by construction

Computational Flow

Processing quantitative tasks involves coordinated interaction between linguistic and mathematical components:

5. **Problem Encoding:** Transformer layers parse problem description, identifying physical domain, relevant variables, and constraint structure
6. **Feature Extraction:** Learned feature engineering (analogous to CCA) reduces high-dimensional state space to characteristic parameters
7. **Mathematical Kernel Invocation:** AGM-elliptic integral layers compute transfer functions from extracted features—actual numerical operations, not simulation
8. **Invariant Analysis:** Extract continuous (ξ) and discrete (S) diagnostics from computed transfer functions
9. **Result Synthesis:** Transformer layers interpret mathematical outputs, generating human-readable explanations with quantitative backing

This architecture enables the model to perform genuine computation while maintaining interpretability—users can inspect both the linguistic reasoning and the mathematical calculations.

Training Paradigm

Training combines three data sources:

- **Linguistic Corpora:** Scientific literature, textbooks, technical documentation for verbal reasoning
- **Symbolic Mathematics:** Theorem databases, proof repositories, symbolic manipulation datasets
- **Numerical Benchmarks:** Validated physics simulations, experimental datasets with known conservation constraints

The mathematical kernel layers require minimal training—their structure embeds centuries of mathematical insight. Primary learning focuses on feature extraction (mapping problems to appropriate kernel parameters) and result interpretation.

VI. Implementation Roadmap

Phase 1: Proof of Concept (6-12 months)

Objective: Demonstrate feasibility of hybrid architecture on constrained domain

- Implement AGM-elliptic integral layers with automatic differentiation support
- Train on single domain (e.g., gasification systems where framework is validated)
- Benchmark against pure transformer baseline and pure numerical solver
- Validate conservation enforcement and computational efficiency claims

Success Metrics:

- $10^4\times$ speedup versus numerical solver baseline
- Zero conservation violations (architectural guarantee)
- Correlation between predicted invariants (ξ , S) and experimental observables > 0.85

Phase 2: Domain Expansion (12-24 months)

Objective: Demonstrate domain agnosticism and transfer learning

- Extend to trans-omics applications (metabolic networks, protein folding)
- Add manufacturing optimization (semiconductor, battery, display)
- Incorporate financial modeling (derivatives pricing, risk assessment)
- Develop transfer learning protocols—leverage invariant structure across domains

Success Metrics:

- Successful application to at least 3 distinct physical domains
- Transfer learning reduces training data requirements by $10\times$ for new domains
- Maintained computational efficiency (10^4 – $10^6\times$ speedup) across applications

Phase 3: Full Integration (24-36 months)

Objective: Deploy production-ready quantitative LLM with multi-modal capabilities

- Integrate verbal, symbolic, and numerical reasoning in unified architecture
- Scale to billions of parameters with distributed training infrastructure
- Develop specialized interfaces for scientific, engineering, and financial applications
- Establish validation framework for regulatory compliance and safety certification
- Create API ecosystem for third-party integration

Success Metrics:

- Deployment in at least one industrial production environment
- Demonstrable ROI through reduced computational costs or improved process efficiency
- Publication of architectural specifications and validation benchmarks

VII. Target Applications and Impact

Immediate Applications

Domain	Current Challenge	Quantitative LLM Solution
Trans-Omics	Integrating 10,000+ genes, 5,000+ proteins, 1,000+ metabolites—computationally intractable for Bayesian inference	Reduce to characteristic features, enforce conservation by construction, achieve $10^4\times$ speedup enabling real-time clinical decisions
Manufacturing	Process optimization requires expensive CFD simulations incompatible with real-time control	Physics-respecting surrogate models enable real-time parameter optimization while guaranteeing material balance
Energy Systems	Grid optimization with renewables requires rapid scenario evaluation under conservation constraints	Validated framework demonstrates mass/energy closure with $10^5\times$ speedup versus traditional solvers
Finance	Monte Carlo simulations for derivative pricing computationally expensive with no-arbitrage constraints	Transfer function formulation can encode arbitrage-free constraints while accelerating scenario generation

Transformative Potential

Quantitative LLMs would fundamentally change how we approach computational science:

- **Democratic Access:** Researchers without computational physics expertise could leverage sophisticated numerical methods through natural language interfaces
- **Rapid Prototyping:** Explore parameter spaces and design alternatives orders of magnitude faster than current workflows
- **Interpretable AI:** Physics-informed constraints provide mechanistic understanding absent in pure ML approaches
- **Cross-Domain Innovation:** Universal mathematical architecture enables transfer of insights across traditionally siloed disciplines
- **Regulatory Compliance:** Conservation guarantees and interpretability align with requirements for safety-critical systems

VIII. Consortium and Resource Requirements

Required Expertise

Successful development requires a multidisciplinary consortium spanning software, hardware, and domain sciences:

- **Mathematical Foundations:** Special functions, elliptic integrals, transfer function theory, numerical analysis, constraint satisfaction
- **Machine Learning Architecture:** Transformer design, automatic differentiation, distributed training, hybrid neural-symbolic systems
- **Hardware Architecture:** Event-driven computing, binary state machine design, EPU microarchitecture, synchronization primitives, ASIC design and fabrication (7nm/5nm processes)
- **Domain Applications:** Computational biology, process engineering, computational physics, quantitative finance
- **Validation and Benchmarking:** Experimental datasets, industrial process data, regulatory frameworks, conservation law verification
- **Software Engineering:** High-performance computing, EPU compiler development, event-driven OS design, API architecture, production deployment

Computational Infrastructure

Quantitative LLM development requires a three-tier computational infrastructure reflecting the hybrid architecture:

Phase 1 requirements (proof of concept):

- 100-500 GPU/NPU cluster for transformer training (A100 or H100 equivalents)
- Specialized mathematical accelerators for AGM-elliptic integral evaluation
- EPU simulator implementation and validation against CPU/GPU baselines
- 10-50 TB storage for datasets, model checkpoints, and constraint validation logs
- High-bandwidth interconnect for heterogeneous processing coordination

Phase 2 requirements (hardware prototyping):

- EPU test chip fabrication (7nm process, 4-8 cores)
- Hybrid NPU-EPU development board for architecture validation
- Event Dispatch Network (EDN) interconnect prototyping
- Power measurement infrastructure for efficiency validation

Phase 3 requirements (production deployment):

- 1,000+ NPU training cluster for full-scale model development
- Production EPU arrays (16-32 cores per chip, 100+ chips per inference server)
- 100+ TB distributed storage with event-aware caching optimization
- Hybrid inference infrastructure optimizing NPU-EPU task allocation
- Software stack including EPU compiler, event-driven OS, and constraint debugging tools

Funding Strategy

Three-tier funding approach balancing public research, industry partnership, and venture investment:

Tier 1: Public Research Grants

Target NSF, DOE, NIH programs focused on computational methods, AI for science, and high-dimensional data integration. Emphasize fundamental research contributions and methodological innovation.

Tier 2: Strategic Industrial Partnerships

Engage Samsung (manufacturing optimization), pharmaceutical companies (drug discovery), energy companies (process optimization). Structure as cost-sharing arrangements where partners provide computational resources, domain expertise, and validation data in exchange for IP rights and first deployment opportunities.

Tier 3: Venture Capital

After proof-of-concept validation, seek Series A funding from AI-focused venture firms. Position as infrastructure play—platform enabling next generation of scientific computing across multiple sectors.

IX. Conclusion: The Quantitative Imperative

Current large language models represent a remarkable achievement in linguistic intelligence but remain fundamentally limited to verbal reasoning. As AI systems increasingly engage with quantitative domains—from drug discovery to climate modeling to financial risk assessment—the absence of authentic numerical reasoning becomes a critical bottleneck.

The path forward requires more than incremental improvements to token-based architectures. It demands a fundamental reconceptualization at both the algorithmic and hardware levels: integrating physics-informed mathematical kernels as computational primitives within LLM architectures, enabled by specialized hardware that treats constraint satisfaction as a native operation.

The AGM-based elliptic integral framework provides the algorithmic blueprint. The Event Processing Unit architecture provides the hardware substrate. Together, they represent a complete solution—mathematics and silicon co-designed for the quantitative reasoning challenge.

The demonstrated ability of the AGM framework to achieve mass conservation, computational efficiency, and physical interpretability across validated test cases confirms algorithmic viability. The EPU's architectural specialization for binary state transitions and constraint enforcement addresses the hardware bottleneck that would otherwise limit deployment. These are not theoretical possibilities but concrete designs ready for implementation.

The opportunity is singular: building the first quantitative LLM would not merely advance AI capabilities—it would democratize sophisticated numerical methods, accelerate scientific discovery across disciplines, and establish a new paradigm for human-AI collaboration in quantitative domains.

The technical foundation exists. The computational infrastructure is achievable. The scientific need is urgent. What remains is the commitment to pursue this transformative vision.

The age of purely verbal AI is giving way to an era of quantitative intelligence. The question is not whether this transition will occur, but who will lead it.